

Foundations of machine learning

Large Language Models and Transformers

Maximilian Kasy

Department of Economics, University of Oxford

Winter 2025

Outline

- Natural language model as a prediction problem.
- Self-supervised learning.
- Self-attention.
- Transformer models.
- Generative AI and beam search.

Takeaways for this part of class

- Natural language models are joint probability distributions for sequences of tokens $(\mathbf{X}_1, \mathbf{X}_2, \dots)$.
- Tasks such as translation or question answering are based on conditional probability distributions of sequences $(\mathbf{Y}_1, \mathbf{Y}_2, \dots)$ given $(\mathbf{X}_1, \mathbf{X}_2, \dots)$.
- Self-supervised learning predicts tokens $\mathbf{X}_t$ given their context $\dots, \mathbf{X}_{t-1}, \mathbf{X}_{t+1}, \dots$.
- A successful class of models uses self-attention, stacked into multiple layers. Such models are called Transformers.
- Generative AI is based on sequential prediction of $\mathbf{X}_t$ given $(\mathbf{X}_1, \dots, \mathbf{X}_{t-1})$. Finding high-probability predicted sequences often uses beam-search.

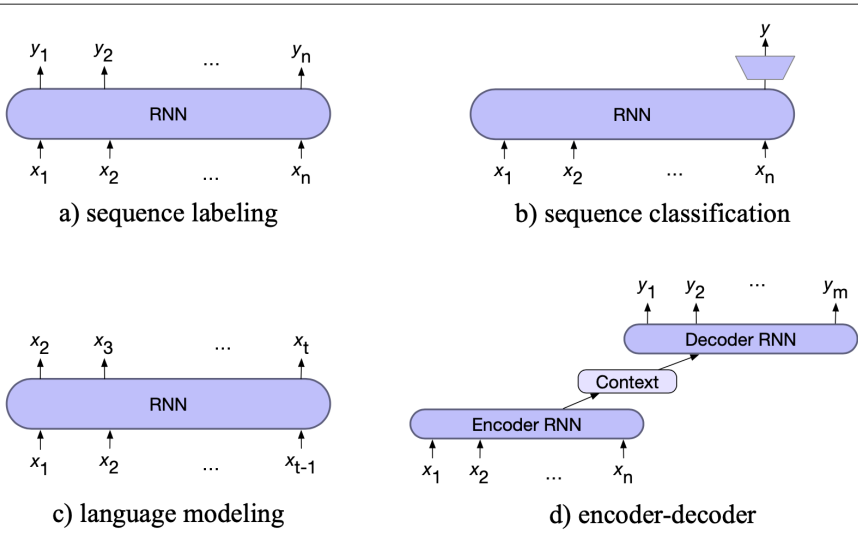


Figure 9.15 Four architectures for NLP tasks. In sequence labeling (POS or named entity tagging) we map each input token x_i to an output token y_i . In sequence classification we map the entire input sequence to a single class. In language modeling we output the next token conditioned on previous tokens. In the encoder model we have two separate RNN models, one of which maps from an input sequence $\mathbf{x}$ to an intermediate representation we call the **context**, and a second of which maps from the context to an output sequence $\mathbf{y}$.

Prediction tasks in language processing

The transformer architecture

Generative AI

References

Prediction tasks in language processing

- Suppose the data consist of pairs of sequences of “tokens.”
 $\mathbf{x} = (x_1, \dots, x_n)$ and $\mathbf{y} = (y_1, \dots, y_m)$.
- Various tasks in language processing require to estimate models $\hat{P}$ for

$$P(Y|X)$$

- Typical loss function for an observation $(\mathbf{x}, \mathbf{y})$: Negative log likelihood,

$$-\log \hat{P}(Y = \mathbf{y} | X = \mathbf{x}).$$

Examples:

1. Machine translation:
 $\mathbf{x}$ is a sentence in the source language.
 $\mathbf{y}$ is a sentence in the target language.
2. Question answering:
 $\mathbf{x}$ is a question. $\mathbf{y}$ is an answer.

Self-supervised learning

- These prediction problems require specific data – pairs of $\mathbf{x}$ and $\mathbf{y}$.
- There is much greater availability of data of “unlabeled” sequences $\mathbf{x}$.
E.g., all the text on the internet (Wikipedia, Arxiv, Github, ...).
- Self-supervised learning fits models for the distribution of such sequences.

Leading cases:

1. Autoregressive models:
Model $P(\mathbf{x}_i | \mathbf{x}_1, \dots, \mathbf{x}_{i-1})$, for all i .
2. Masking:
Model $P(\mathbf{x}_i | \mathbf{x}_1, \dots, \mathbf{x}_{i-1}, \mathbf{x}_{i+1}, \dots, \mathbf{x}_n)$, for all i .

Masking

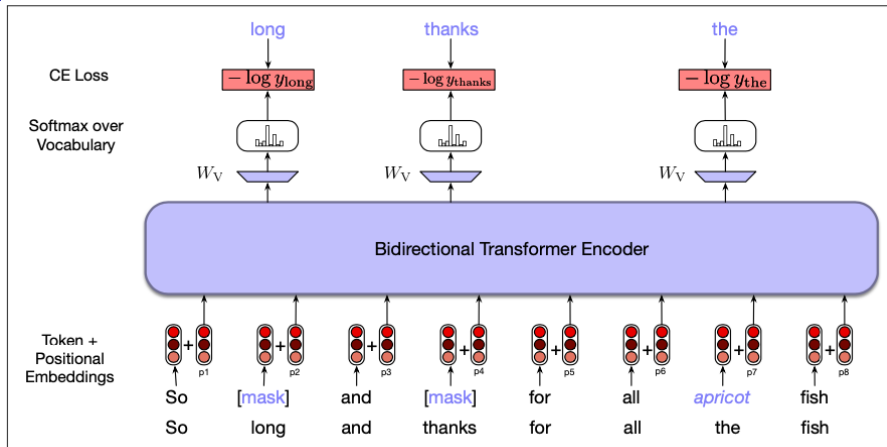


Figure 11.5 Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. (In this and subsequent figures we display the input as words rather than subword tokens; the reader should keep in mind that BERT and similar models actually use subword tokens instead.)

Embeddings and pre-training

- Many language models are trained in two steps:
 1. *Self-supervised learning* on a large corpus of sequences $\mathbf{x}$, using masking.
This yields an embedding (representation) of the source data $\mathbf{x}$.
 2. *Fine-tuning* on a task-specific corpus:
Using the embeddings from 1. as predictors for $\mathbf{y}$.
- This is also known as *transfer learning*.
It yields much better results than simply training on the task-specific corpus.

Prediction tasks in language processing

The transformer architecture

Generative AI

References

The transformer architecture

- How do we get an embedding for a sequence of tokens?
- What functional form should we choose?
- Leading answer: *Transformers*.
- Transformers consist of multiple transformer blocks.
- Each of which includes self-attention layers.

Self-attention layers

- Take as given a sequence of input vectors $\mathbf{x}_1, \dots, \mathbf{x}_n$,
- We want to *transform it*,
to produce a sequence of output vectors $\mathbf{y}_1, \dots, \mathbf{y}_n$
of the same dimension.
- $\mathbf{y}_j$ is supposed to encode the meaning of $\mathbf{x}_i$ *in the context* of the other $\mathbf{x}_j$.
- First step: Take a linear transformation of the $\mathbf{x}_i$.

$$\mathbf{v}_i = \mathbf{W}^v \cdot \mathbf{x}_i.$$

- Second step: Take a weighted average of the $\mathbf{v}_i$ to get the output $\mathbf{y}_i$.

$$\mathbf{y}_i = \sum_j \alpha_{ij} \mathbf{v}_j.$$

Self-attention layers continued

- The weights α_{ij} capture the importance of x_j as context for x_i .
- But where do the weights come from? *Self-attention!*

$$\alpha_{ij} = \frac{\exp(\text{score}_{ij})}{\sum_{j'} \exp(\text{score}_{ij'})}.$$

Normalizing sum of weights to **1** (aka softmax / multinomial logit).

- **score_{ij}**: Relevance of x_j as context for x_i .

$$\text{score}_{ij} = \langle q_i, k_j \rangle \quad \text{inner product}$$

$$q_i = W^q \cdot x_i \quad \text{query}$$

$$k_j = W^k \cdot x_j \quad \text{key}$$

- Contrast to time series models: Weights depend only on $|i - j|$.
⇒ Would not recognize importance of far-away sentence parts for context.

Backward looking and bi-directional self-attention

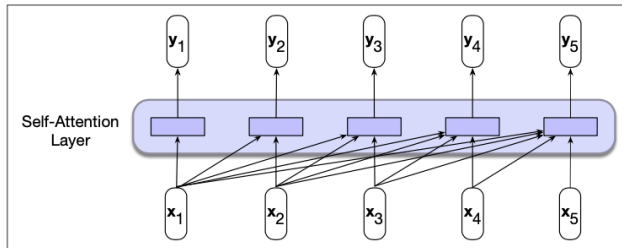


Figure 11.1 A causal, backward looking, transformer model like Chapter 10. Each output is computed independently of the others using only information seen earlier in the context.

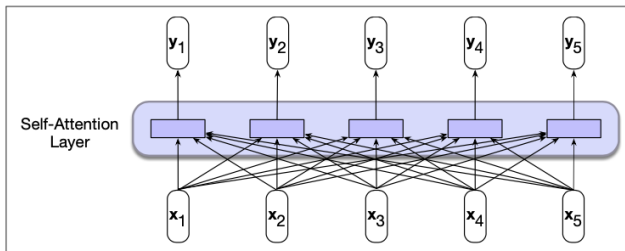


Figure 11.2 Information flow in a bidirectional self-attention model. In processing each element of the sequence, the model attends to all inputs, both before and after the current one.

Transformer blocks

- Self-attention layers are packaged with some additional transformations as follows:

$$z = \text{LayerNorm}(x + \text{SelfAttention}(x))$$

$$y = \text{LayerNorm}(z + \text{FFN}(z))$$

- $\text{LayerNorm}(x)$ normalizes $x = (x_1, \dots, x_n)$ by subtracting the mean and dividing by the standard deviation.
- The addition of x to $\text{SelfAttention}(x)$ is called “residual connection.” This keeps raw information from previous input.
- $\text{FFN}(z)$ is a standard feed-forward neural network.

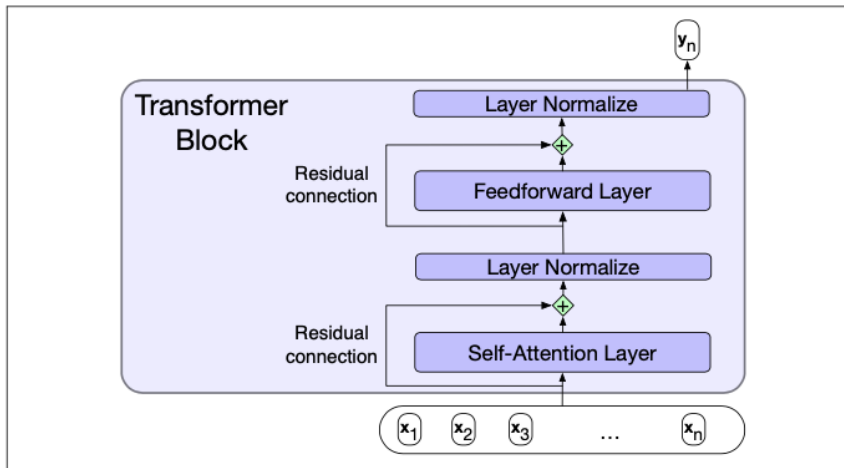


Figure 10.4 A transformer block showing all the layers.

Multi-head attention

- Tweak on the transformer block:
Replace the single self-attention layer
by several parallel versions, indexed by b .
- Thus:

$$y_i^b = \sum_j \alpha_{ij}^b \cdot [W^{v,b} \cdot x_j], \quad \alpha_{ij}^b = \frac{\exp(\text{score}_{ij}^b)}{\sum_{j'} \exp(\text{score}_{ij'}^b)},$$
$$\text{score}_{ij}^b = \left\langle [W^{q,b} \cdot x_i], [W^{k,b} \cdot x_j] \right\rangle.$$

- The rest of the transformer block stays the same.
- Motivation: Context matters in various ways.

Prediction tasks in language processing

The transformer architecture

Generative AI

References

Generative AI

- Suppose you have fit an autoregressive model, which gives

$$\hat{P}(y_{i+1}|\mathbf{x}, y_1, \dots, y_{i-1}).$$

- Suppose you would like to generate a prediction of $\mathbf{y}$, given an input $\mathbf{x}$.
- That is you would like to find

$$\hat{\mathbf{y}} = \operatorname{argmax}_{\mathbf{y}} \hat{P}(\mathbf{y}|\mathbf{x}) = \operatorname{argmax}_{\mathbf{y}} \prod_i \hat{P}(y_i|\mathbf{x}, y_1, \dots, y_{i-1}).$$

- Such forecasting of autoregressive models is at the heart of “generative AI.”

Greedy sampling

- Naive idea: Sequentially find the highest probability prediction, one step at a time:

$$\hat{y}_i = \operatorname{argmax} \hat{P}(y_{i+1}|x, y_1, \dots, y_{i-1}).$$

- This is known as greedy search.
- Problem:
This does not take into account the impact of the choice of $\hat{y}_i$ on the availability of high probability choices later.
- Dynamic programming problem!

Beam search

- Exhaustive search of the tree of possible sequences is too costly.
- Compromise: Beam search.
 1. Start with the k highest-probability choices for $\hat{y}_1$.
 2. For each of these choices separately, find the k highest probability choices for $\hat{y}_2$.
 3. Keep the k sequences of $\hat{y}_1, \hat{y}_2$ with the highest probability, discard the rest.
 4. Iterate.

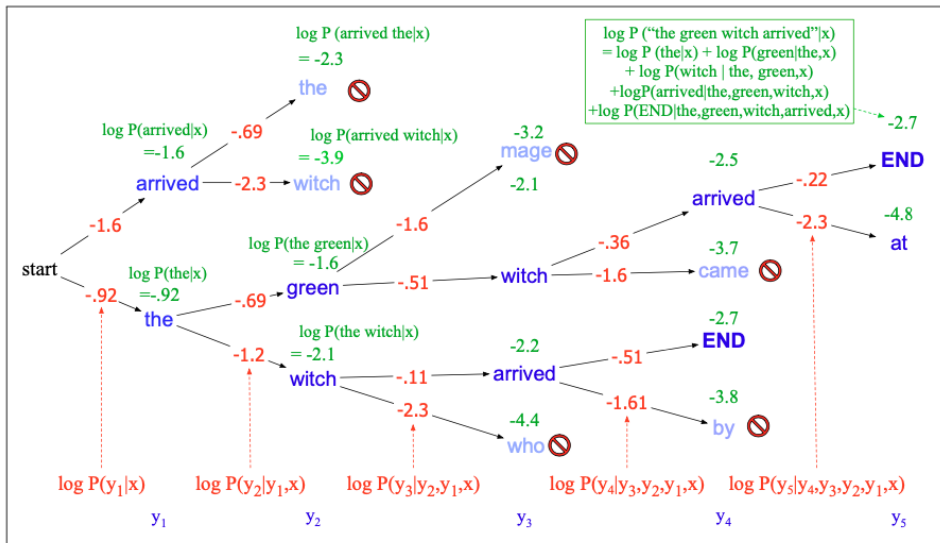


Figure 10.10 Scoring for beam search decoding with a beam width of $k = 2$. We maintain the log probability of each hypothesis in the beam by incrementally adding the logprob of generating each next token. Only the top k paths are extended to the next step.

References

Speech and Language Processing,
Dan Jurafsky and James H. Martin, 2023,
chapters 10-11.